



Cursus Data Engineer

Rapport de projet PREDICTELEC

George CARDENAS
Julien NEVILLE
Caroline NONY

Mai 2026

1. Contexte du projet

La production d'électricité par les énergies renouvelables (éolien, photovoltaïque), par nature intermittentes, pose des difficultés importantes aux gestionnaires de réseaux électriques. En effet, la stabilité du réseau nécessitant à tout instant l'équilibre entre la puissance produite et la puissance consommée, les pics et les creux intempestifs dus aux renouvelables doivent être compensés en temps réel. Cela peut se faire de deux façons :

- en modulant la puissance produite par les centrales pilotables (centrales nucléaires ou hydroélectriques principalement en France) ;
- en modulant la puissance consommée par des consommateurs pilotables (démarrables ou arrêtables sur demande).

En France, la modulation est aujourd'hui réalisée principalement par les centrales nucléaires qui ont l'obligation légale de réduire leur puissance en cas de pic de production des renouvelables, ce qui a des conséquences néfastes sur la rentabilité et sur la durée de vie des réacteurs nucléaires.

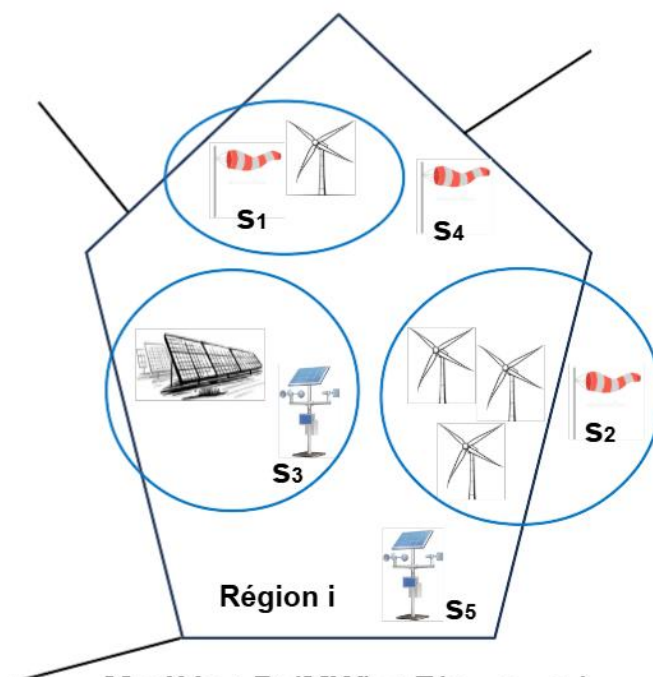
Ainsi, partout dans le monde, on cherche à développer des parcs de consommateurs pilotables. Par exemple, le 11 juillet 2025, la proposition de loi N° 1750 (https://www.assemblee-nationale.fr/dyn/17/textes/l17b1750_proposition-loi) a été déposée à l'Assemblée Nationale pour autoriser à titre expérimental l'utilisation des surplus électriques pour le minage de cryptoactifs. Assurer la disponibilité des consommateurs pilotables lors des pics de production est alors crucial, non seulement pour leur propre rentabilité, mais aussi pour la gestion du réseau dans son ensemble.

Le projet consiste donc à développer une application data permettant de prédire la production électrique d'origine éolienne et photovoltaïque sur le territoire français à partir des prévisions météorologiques.

2. Principe de la solution

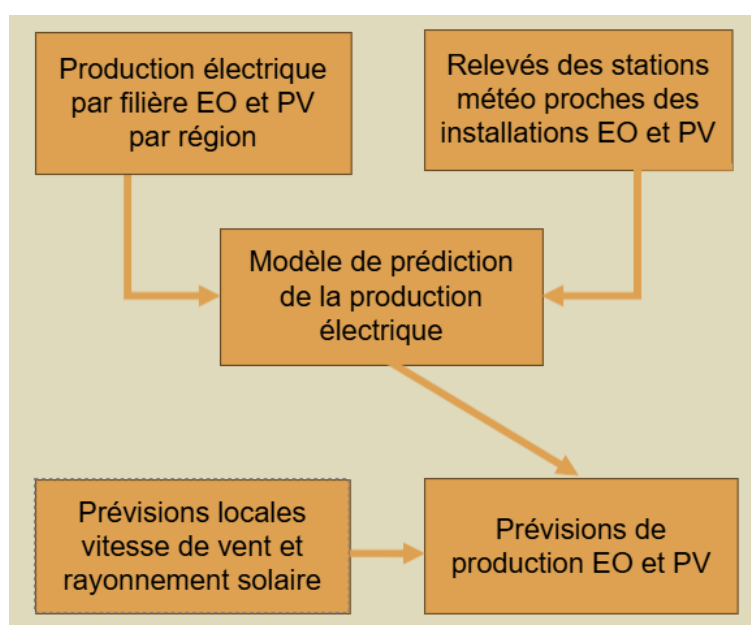
On va créer un modèle de prédiction de la production électrique d'origine éolienne (EO) et photovoltaïque (PV) à partir des prédictions météorologiques à l'échelle de la France métropolitaine continentale, qui forme un réseau électrique interconnecté. Ce modèle sera entraîné avec les données historiques :

- de la puissance électrique en mégawatts (MW) produite par les filières EO et PV, en fonction des heures de la journée et sur plusieurs jours ;
- des relevés météorologiques (force du vent et rayonnement solaire) des stations à proximité des installations EO et PV et sur les mêmes intervalles de temps.

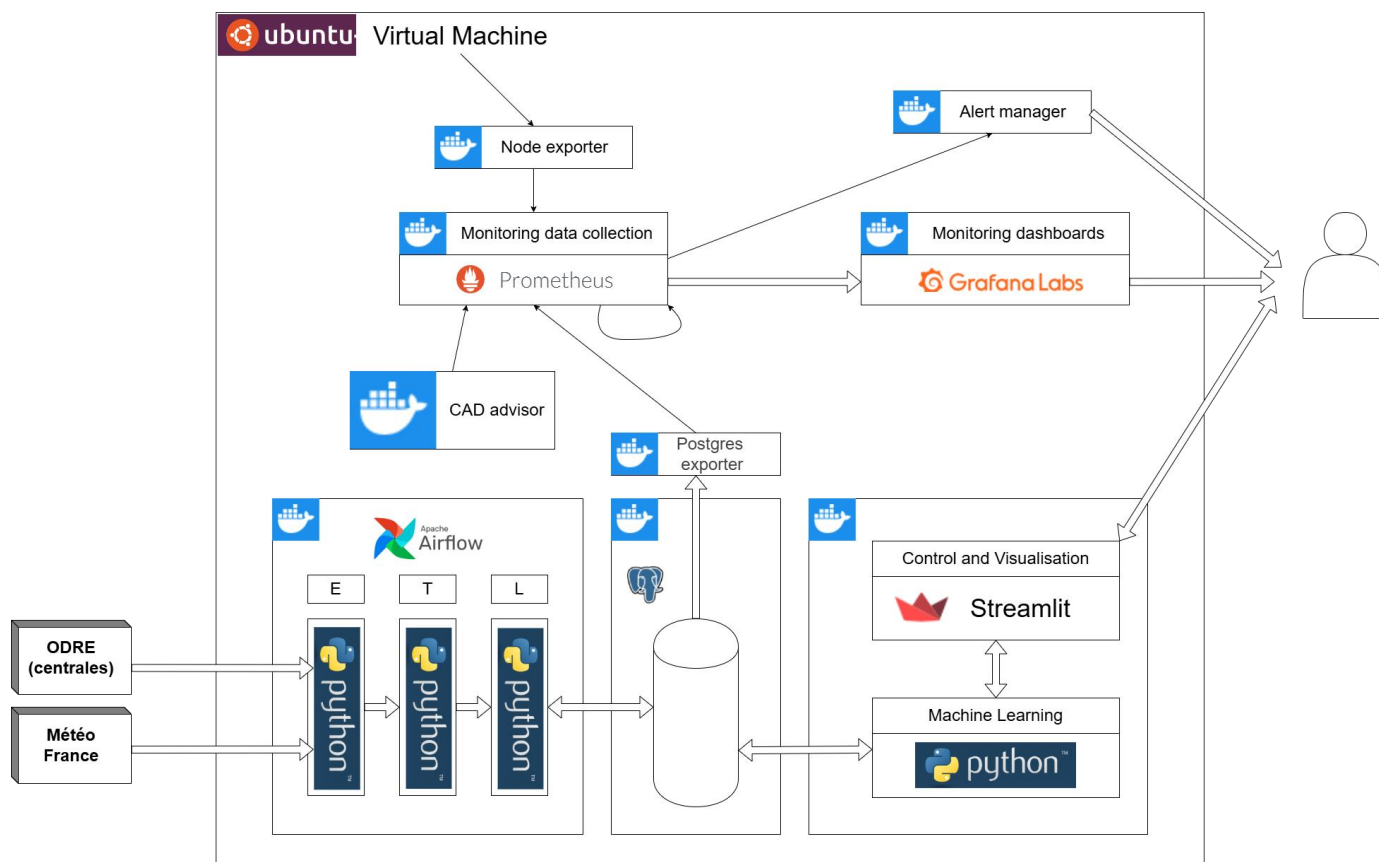


$$\text{Modèle : } P_i \text{ (MW)} = F(s_1, s_2, s_3)$$

$$P_{\text{France}} \text{ (MW)} = \sum P_i$$



3. Architecture générale de l'application



4. Collecte des données

Parmi les données exposées par les API, les données effectivement récupérées pour usage dans l'application sont **surlignées en jaune** dans les différents tableaux.

4.1. Production électrique EO et PV

Opendata Réseaux-Energie (ODRE) met à disposition en licence ouverte des jeux de données de production selon les différentes filières composant le mix énergétique. Deux jeux de données sont utilisés :

- Un jeu de données "temps réel", rafraîchi une fois par heure présente des **données régionales** "temps réel" issues de l'application éco2mix. Elles proviennent des télémesures des ouvrages complétées par des forfaits et estimations.
 - o Site : [Données éco2mix régionales temps réel — Open Data Réseaux Énergies \(ODRE\)](#)
- Un jeu de données "consolidées", qui remplace les données "temps réel" du mois M qui sont alors effacées en milieu de mois M+1
 - o Site : [Données éco2mix régionales consolidées et définitives \(janvier 2013 à janvier 2023\) — Open Data Réseaux Énergies \(ODRE\)](#)

Dans notre application, les données "temps réel" sont mises à jour avec les données "consolidées" quand ces dernières sont disponibles. Les données sont fournies au pas horaire de 15 min.

Les valeurs de production électrique récupérées sont tch_eolien et tch_solaire, qui représentent le taux de charge des filières éoliennes et solaires respectivement, c'est à dire leur volume de production par rapport à la capacité de production installée et en service de cette filière.

Quota de 50000 appels par utilisateur et par mois. Limite de 100 enregistrements retournés par appel d'API.

Appel	Résultat
https://odre.opendatasoft.com/api/explore/v2.1/catalog/datasets/eco2mix-regional-tr/records?where=code_insee_region%3D24&limit=1	<pre>{ "total_count":5760, "results":[{ "code_insee_region":"24", "libelle_region":"Centre-Val de Loire", "nature":"Données temps réel", "date":"2026-02-01", "heure":"00:00", "date_heure":"2026-01-31T23:00:00+00:00", "consommation":2407, "thermique":1, "nucleaire":10630, "eolien":351, "solaire":0, "hydraulique":39, "pompage":0, "bioenergies":30, "ech_physiques":-8645, "stockage_batterie":0, "destockage_batterie":0, "tco_thermique":0.04, "tch_thermique":0.5, "tco_nucleaire":441.63, "tch_nucleaire":91.4, "tco_eolien":14.58, "tch_eolien":19.79, "tco_solaire":0, "tch_solaire":0, "tco_hydraulique":1.62, "tch_hydraulique":41.94, "tco_bioenergies":1.25, "tch_bioenergies":32.97, "column_68":null }] }</pre>

4.2. Installations

4.2.1. Géolocalisation régions et départements

4.2.2. Centrales électriques EO et PV

Opendata Réseaux-Energie (ODRÉ) met à disposition en licence ouverte le registre national des installations de production et de stockage d'électricité avec leur localisation.

- Site: [Registre national des installations de production et de stockage d'électricité \(au 31/01/2026\) — Open Data Réseaux Énergies \(ODRÉ\)](#)

Limite de 100 enregistrements retournés par appel d'API.

Appel	Résultat
https://odre.opendatasoft.com//api/explore/v2.1/catalog/datasets/registre-national-installation-production-stockage-electricite-agrege/records?limit=1	<pre>{ "total_count":131225, "results":[{ "nominstallation":"ALLANE A3 - ADP A3 DE LA FERME EOLIENNE DE ALLANCHE", "codeicresourceobject":"17W100P100P2491R", "codeiris":"151510000", "codeinseecommune":"15151", "commune":"Peyrusse", "codeepci":"200066637", "epci":"CC Hautes Terres Communauté", }] }</pre>

	<pre> "code departement": "15", "departement": "Cantal", "code region": "84", "region": "Auvergne-Rhône-Alpes", "code iris commune implantation": "150010000", "code insee commune implantation": "15001", "codes 3 renr": null, "date raccordement": "01/07/2012", "date de raccordement": null, "date mise en service": "01/08/2012", "date de début version": "01/01/2014", "postesource": "PEYRU", "tension raccordement": "63 kV", "mode raccordement": "DIRECT", "code filière": "EOLIE", "filière": "Eolien", "code combustible": null, "combustible": null, "codes combustibles secondaires": null, "combustibles secondaires": null, "code technologie": "TERRE", "technologie": "Terrestre", "type stockage": null, "puissance installée": 3000, "puissance max charge": null, "puissance max": 3000, "puissance installée en discharge": null, "nb groupes": 1, "nb installations": 1, "regime": "En service", "energie stockable": null, "capacité réservoir": null, "hauteur chute": null, "productible": null, "débit maximal": null, "code gestionnaire": "10XFR-RTE-----Q", "gestionnaire": "RTE", "energie annuelle glissante injectée": 366048 9, "energie annuelle glissante produite": 366048 9, "energie annuelle glissante soutirée": null, "energie annuelle glissante stockée": null, "max puis": 3000, "date mise en service_date": "2012-08-01" }] </pre>
--	--

4.2.3. Stations météo

Le portail des API de Météo France (<https://portail-api.meteofrance.fr>) permet, via l'API PaquetObs, de récupérer au format json les données des stations météo par département : identifiants et localisation géographique via latitude et longitude.

Appel	Résultat
https://public-api.meteofrance.fr/public/DPPaquetObs/v1/paquet/horaire?id-departement=1&format=json	<pre> [{ "lat": 46.278167, "lon": 5.669, "geo_id_insee": "01014002", "reference_time": "2026-04-01T13:10:06Z", "insert_time": "2026-04-01T13:02:15Z", "validity_time": "2026-04-01T13:00:00Z", "t": 284.05, "td": 271.25, "tx": 285.55, "tn": 282.65, "u": 41.0, "ux": 46.0, "un": 37.0, "dd": 350.0, "ff": 7.6, </pre>

	<pre> "dxy": 350.0, "fx": 7.6, "dx": 330.0, "fx": 13.2, "rr": 0.0, "t_10": null, "t_20": null, "t_50": null, "t_100": null, "vv": null, "etat_sol": null, "sss": 0.0, "n": null, "insolh": null, "ray_glo01": null, "pres": null, "pmer": null }, ...] </pre>
--	---

Note : nous passons par cette API, qui fournit non seulement la localisation géographique des stations mais aussi un relevé météo, et non par l'API /liste-stations, car nous souhaitons pouvoir filtrer les stations mettant à disposition les mesures de vent ("ff") et de rayonnement solaire ("ray_glo1"). Nous les appelons "stations éligibles".

Note : l'utilisation des API Météo France nécessite de façon générale une authentification par un token à durée limitée à 1h, ce qui entrave l'automatisation. La solution trouvée à cette limite est présentée dans le paragraphe 3.4.

4.3. Données météorologiques

4.3.1. Relevés météorologiques des stations

Les relevés météo quotidien par station sont obtenus via l'API PaquetObs de Météo France ([METEO FRANCE – L'API DonneesPubliquesPaquetObservation](#)). Pour respecter la limite de 100 requêtes par minutes, les données sont extraites par batch de stations avant d'être chargées en base de données.

Appel	Résultat
<pre> https://public- api.meteofrance.fr/public/DPPaquetObs/v1/paquet/infrahoraire- 6m?id_station=75107005&format=json' </pre>	<pre> [{ "lat": 48.858333, "lon": 2.2945, "geo_id_insee": "75107005", "reference_time": "2026-04-20T09:42:08Z", "insert_time": "2026-04-20T09:36:35Z", "validity_time": "2026-04-20T09:30:00Z", "t": 281.55, "td": null, "u": null, "dd": 30, "ff": 6.1, "dxi10": 50, "fxi10": 8.9, "rr_per": null, "t_10": null, "t_20": null, "t_50": null, "t_100": null, "vv": null, </pre>

	<pre>"etat_sol": null, "sss": null, "insolh": null, "ray_glo01": null, "pres": null, "pmer": null }, ...]</pre>
--	--

Les historiques mensuels de relevés météo par station sont obtenus via l'API des Données Climatologiques de Météo France (<https://portail-api.meteofrance.fr/web/fr/api/DonneesPubliquesClimatologie>). Pour chaque station, les données sont obtenues en deux temps par un mécanisme asynchrone :

1. Commande des données climatologiques d'une station sur toute la période voulue (début_api / fin_api).
Appel : `https://public-api.meteofrance.fr/public/DPCLim/v1/commande-station/horaire?id-station={station}&date-deb-periode={debut_api}&date-fin-periode={fin_api}`
2. Récupération des données liées à la commande.
Appel : `https://public-api.meteofrance.fr/public/DPCLim/v1/commande/fichier?id-cmde={commande}`

Pour chaque station, la période de récupération est divisée par plages de 30 jours, conduisant à autant de commandes que de plages.

4.3.2. Prévisions météo

Les prévisions météorologiques sont fournies par l'API Arpège de Météo France (<https://portail-api.meteofrance.fr/web/fr/api/ARPEGE>), qui fournit des prévisions détaillées à l'échelle locale jusqu'au jour J+4. La récupération des données se fait en trois temps :

1. Récupération, avec l'endpoint GetCapabilities, des codes d'identification des modèles de calculs Arpège pour, respectivement, la vitesse du vent et pour le rayonnement solaire ("coverage_id"). Un nouveau modèle est calculé toutes les 6h et les modèles sont horodatés : on récupère donc le dernier modèle disponible pour chacun des paramètres météo. La réponse est envoyée sous format xml.

Appel	Résultat
<code>https://public-api.meteofrance.fr/public/arpege/1.0/wcs/MF-NWP-GLOBAL-ARPEGE-025-GLOBE-WCS/GetCapabilities?service=WCS&version=2.0.1&language=eng</code>	<pre>... <wcs:CoverageSummary> <ows:Title>Wind speed at specified height level above ground</ows:Title> <wcs:CoverageId>WIND_SPEED__SPECIFIC_HEIGHT_LEVEL_ABOVE_GROUND__2026-04-01T12.00.00Z</wcs:CoverageId> <wcs:CoverageSubtype>ReferenceableGridCoverage</wcs:CoverageSubtype> </wcs:CoverageSummary> <wcs:CoverageSummary> <ows:Title>Wind speed at specified height level above ground</ows:Title> <wcs:CoverageId>WIND_SPEED__SPECIFIC_HEIGHT_LEVEL_ABOVE_GROUND__2026-04-01T18.00.00Z</wcs:CoverageId> <wcs:CoverageSubtype>ReferenceableGridCoverage</wcs:CoverageSubtype> </wcs:CoverageSummary> <wcs:CoverageSummary> <ows:Title>Wind speed at specified height level above ground</ows:Title> <wcs:CoverageId>WIND_SPEED__SPECIFIC_HEIGHT_LEVEL_ABOVE_GROUND__2026-04-02T00.00.00Z</wcs:CoverageId></pre>

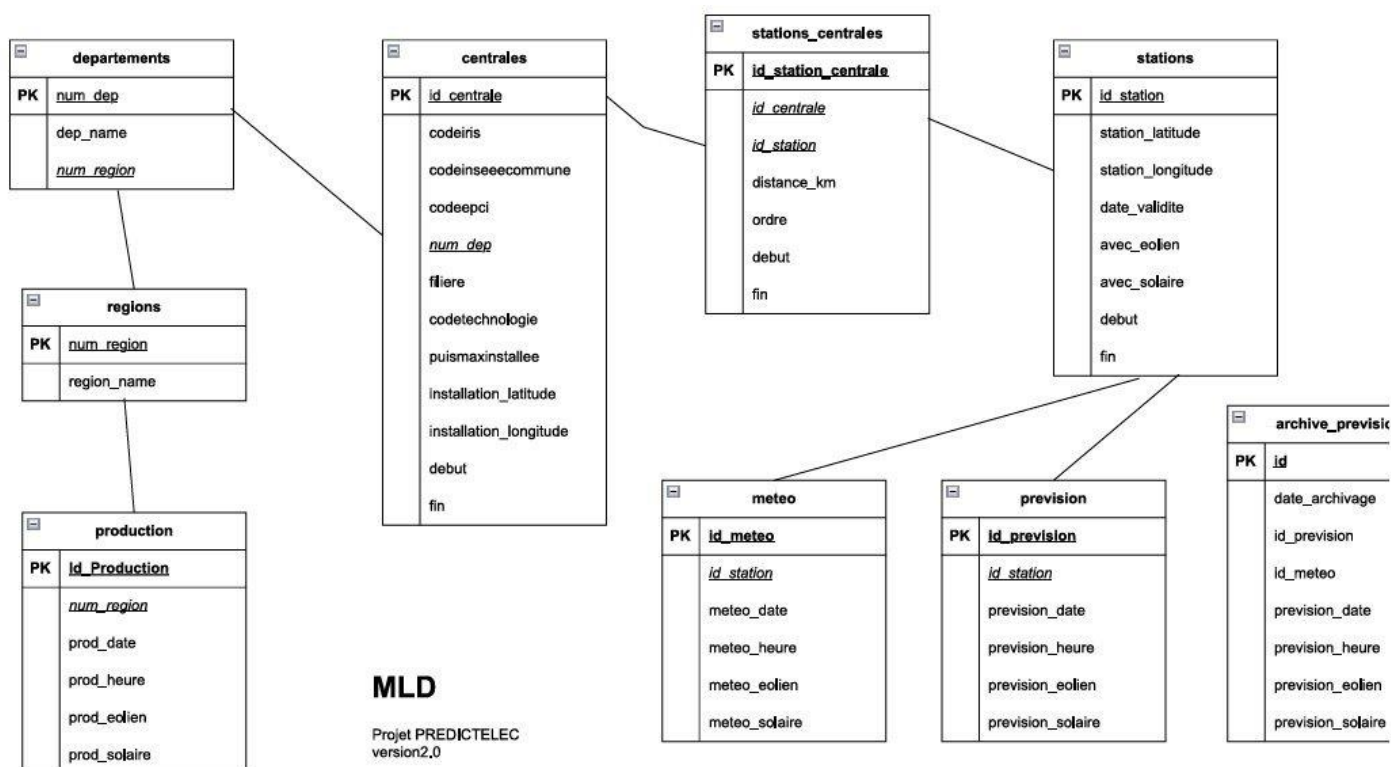
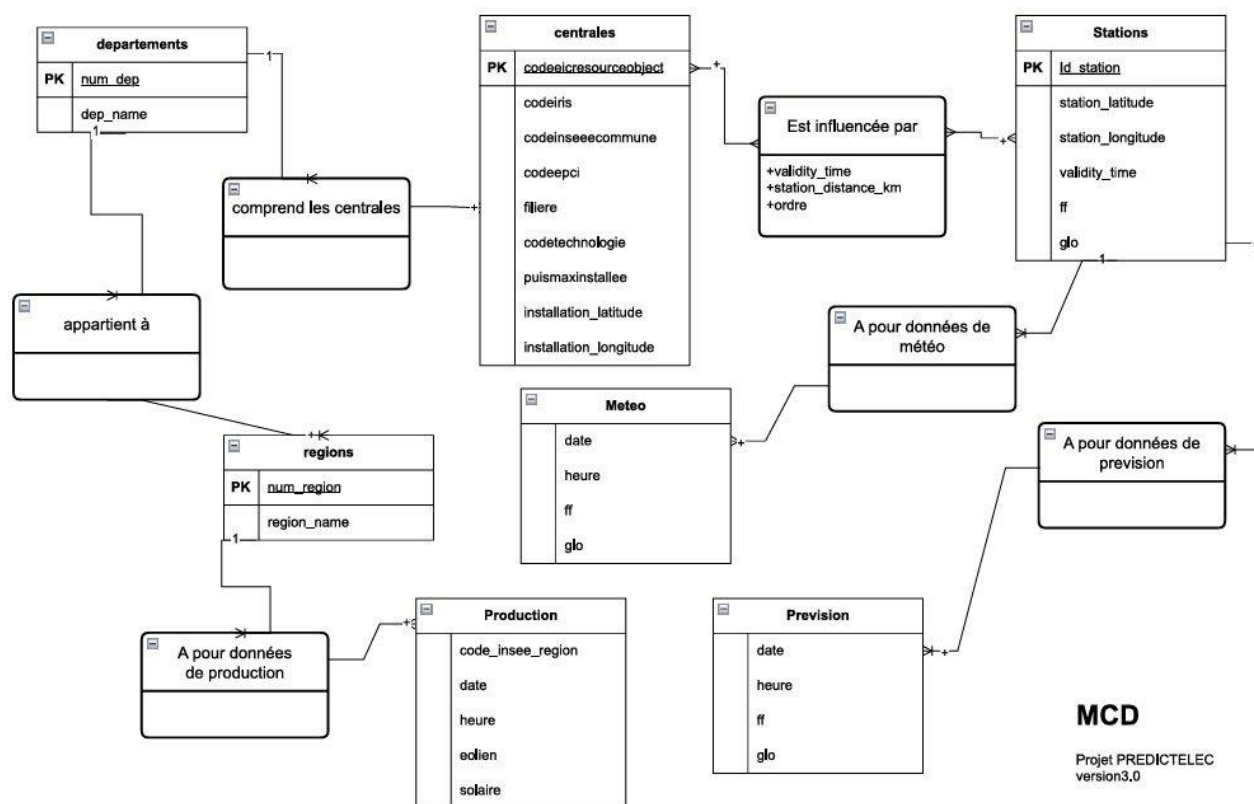
	<pre> <wcs:CoverageSubtype>ReferenceableGridCoverage</wcs:CoverageSub type> </wcs:CoverageSummary> <wcs:CoverageSummary> <ows:Title>Wind speed at isobaric surfaces</ows:Title> <wcs:CoverageId>WIND_SPEED__ISOBARIC_SURFACE__2026-03- 29T00.00.00Z</wcs:CoverageId> <wcs:CoverageSubtype>ReferenceableGridCoverage</wcs:CoverageSub type> </wcs:CoverageSummary> ... </pre>
--	--

2. Récupération, avec l’endpoint DescribeCoverage, des pas de temps de calcul disponibles entre J+1 et J+4, pour chaque “coverage_id”.
3. Récupération, avec l’endpoint GetCoverage, pour chaque “coverage_id”, chaque pas de temps entre J+1 et J+4 et chaque localisation de station météo (repérée par sa latitude et longitude), des prévisions de vitesse de vent et de rayonnement solaire.

Le quota étant de 100 requêtes par minute, nous avons créé différents tokens d’utilisateur de manière à récupérer les données dans un temps raisonnable (~1h).

4.4. Modèle de données

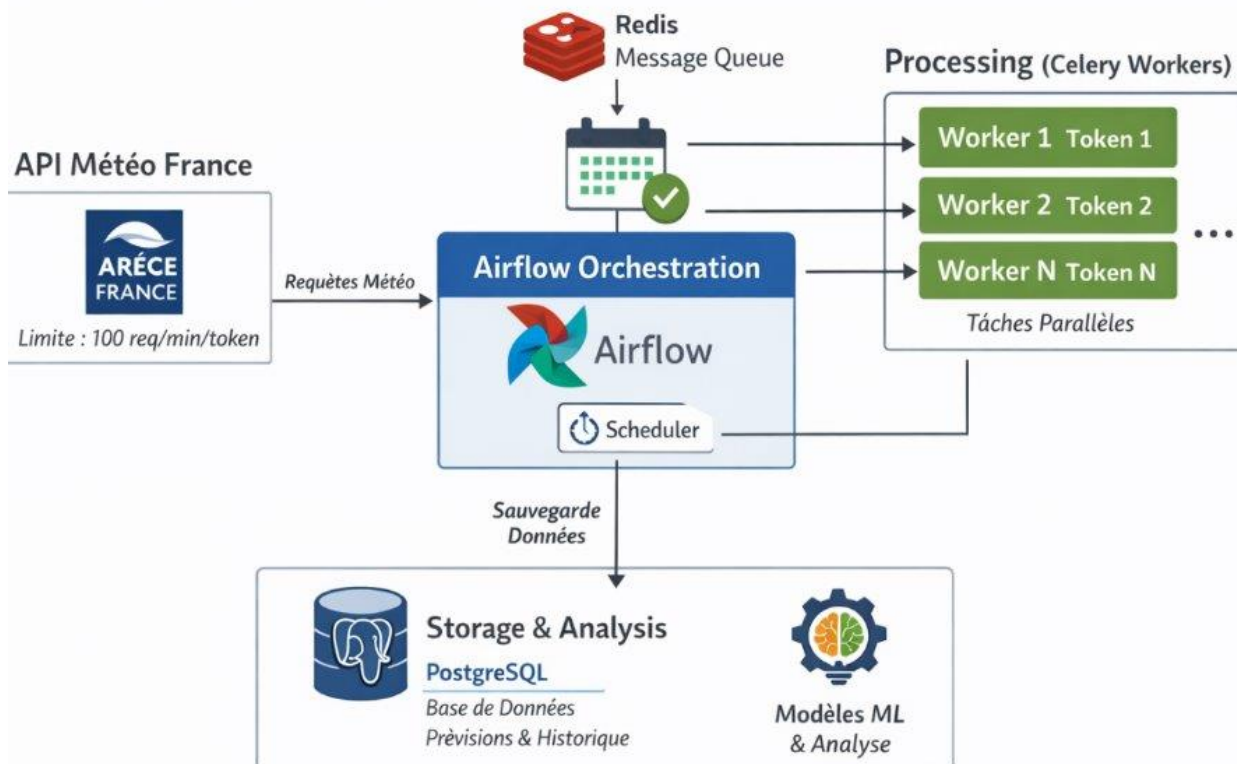
Nous avons fait le choix d'organiser les données dans une base relationnelle avec PostgreSQL.
Ci-dessous les modèles conceptuels et logiques de données adoptés.



4.5. Pipeline ETL et orchestration

L'extraction, la transformation et le chargement des données en base est orchestré au moyen de 3 DAGs Airflow. L'architecture du pipeline ETL est représentée dans la figure ci-dessous.

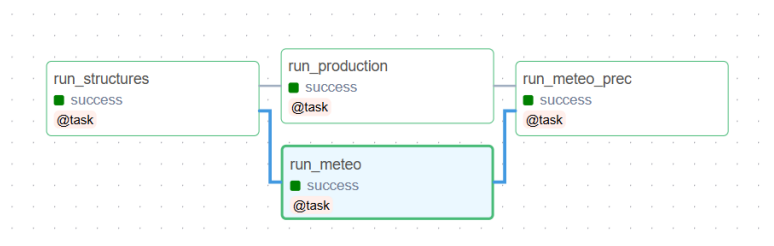
Architecture du Pipeline Predictelec



4.5.1. pipeline_etl_globale

Ce DAG, exécuté 1 fois par mois, lance 4 tasks :

- **run_structures** : lance les scripts pour la récupération des données des installations (centrales électriques et stations météo), leur géolocalisation et l'association des centrales électriques avec les stations météo les plus proches. Pour les centrales, le processus d'extraction est exécuté par batch de 100 installations pour respecter les limites de l'API.
- **run_production** : lance les scripts pour la récupération des données de production électrique par région. Les données "temps réel" (non-consolidées) sont récupérées, ainsi que les nouvelles données consolidées dès elles sont disponibles.
- **run_meteo** : lance les scripts pour la récupération des données de relevés météo du jour et met à jour la vue correspondante en base de données.
- **run_meteo_prec** : lance les scripts pour la récupération des données de relevés météo du mois précédent et met à jour la vue correspondante en base de données.



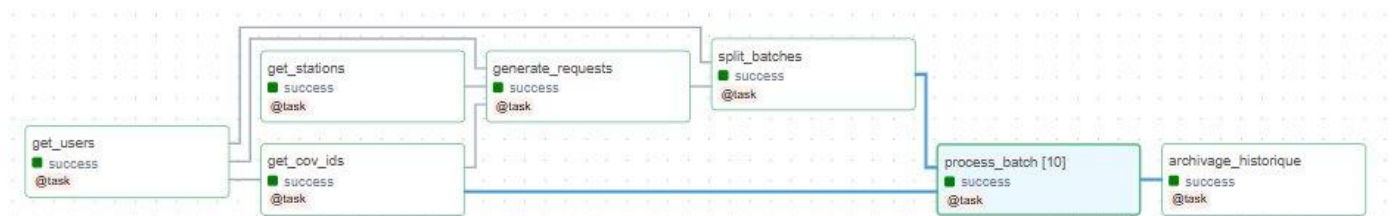
4.5.2. pipeline_etl_majdata

Ce DAG, exécuté quotidiennement, lance uniquement les 2 tasks run_production et run_meteo décrites plus haut.

4.5.3. dag_prevision_v3

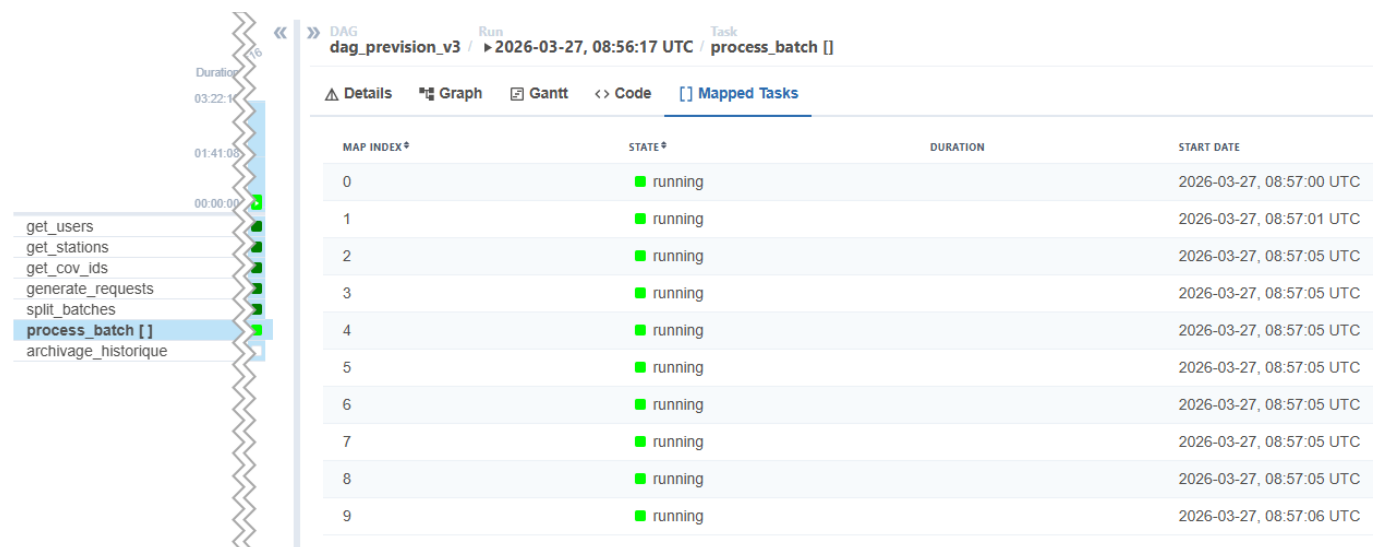
Ce DAG, exécuté quotidiennement, lance 7 tasks:

- get_users : extrait depuis le fichier d’environnement .env de l’application les données d’authentification des utilisateurs enregistrés pour l’API Météo France.
- get_stations : extrait depuis la base les données des stations météo associées aux centrales.
- get_cov_ids : récupère les “coverage_ids” des modèles de prévisions pour la vitesse de vent et le rayonnement solaire.
- generate_requests : génère les paramètres des requêtes des prévisions météo pour chaque station et chaque time_step.
- split_batches : dispatch les paramètres de requêtes (time_step et station) entre les différents users.
- process_batch : pour chaque user et sa liste de paramètres de requêtes (time_step et station), récupère les prévisions de vent et de rayonnement solaire
- archivage_historique : archive les prévisions météo historiques



Le volume de données à extraire quotidiennement pour les prévisions météo étant important, nous avons mis en place plusieurs utilisateurs exécutant les requêtes API en parallèle. Cette approche permet de diviser par 10 la durée d’extraction, ramenée à moins de 50 minutes, malgré une limitation de 100 requêtes par minute et par token utilisateur.

Cette optimisation repose sur l’utilisation de workers Celery, d’un broker Redis pour la distribution des tâches, ainsi que sur l’application de la fonction expand() d’Airflow lors de l’appel à process_batch, permettant le parallélisme des traitements.



Note :

La durée de validité par défaut des tokens pour accéder aux API Météo France étant de 1h seulement, ceux-ci ne sont pas réutilisables de jour en jour. Nous avons donc développé un script permettant la génération automatique de tokens avant chaque appel à une API Météo France.

4.5.4. init_db

Ce DAG est exécuté une fois après le premier déploiement sur une VM. Il crée la base de données PostgreSQL, les tables et les vues.

5. Machine Learning et Visualisation

La pipeline de Machine Learning est représentée dans la figure ci-dessous.



Le contrôle des paramètres des modèles de Machine Learning et la visualisation des résultats sont réalisés avec Streamlit.

5.1. Préparation des données

Les données sont chargées automatiquement depuis les vues matérialisées de la base SQL. Après nettoyage (suppression des doublons, gestion des valeurs manquantes) et conversion de types (ie timestamp), les données sont séparées en un jeu d'entraînement et un jeu de test (paramétrable).

5.2. Modélisation

Deux modèles de machine learning sont utilisés à titre de comparaison :

- une régression linéaire : simple et interprétable ;
- un Random Forest Regressor : capable de capter les relations complexes et robuste aux interactions ;

Les modèles sont comparés notamment au moyen des indicateurs RMSE, MAE et R^2 .

5.3. Prédiction

L'utilisateur peut entrer des paramètres :

- jour à prédire entre J et J+2 (limité par la disponibilité des prévisions météo dans l'API) ;

- la ou les régions d'intérêt ;
- le modèle retenu : linéaire, random forest ou le meilleur modèle (choix par défaut).

Les résultats fournis sont :

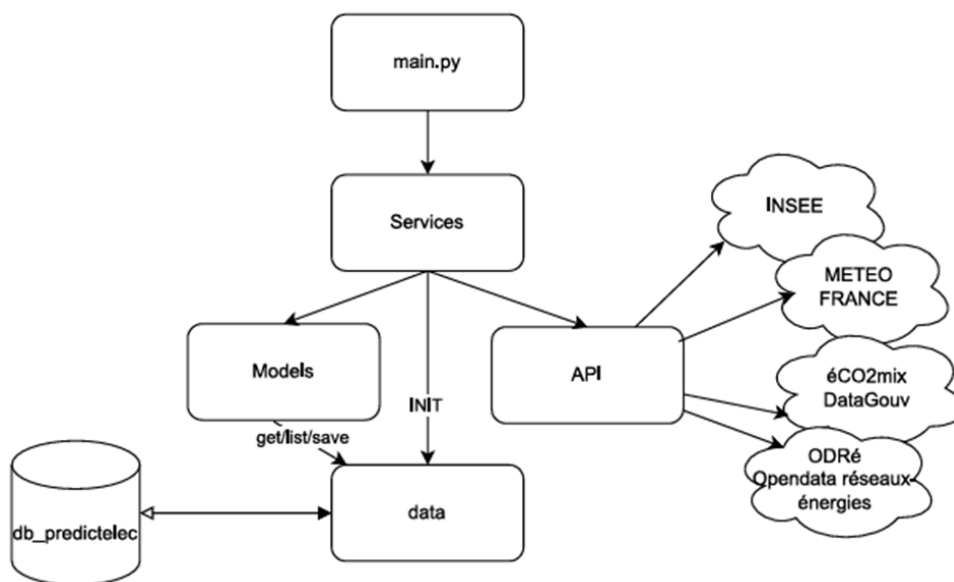
- un tableau de la production prédite par pas de 15 min ;
- l'agrégation journalière par région ;
- la production totale journalière.

Des visualisations sont disponibles :

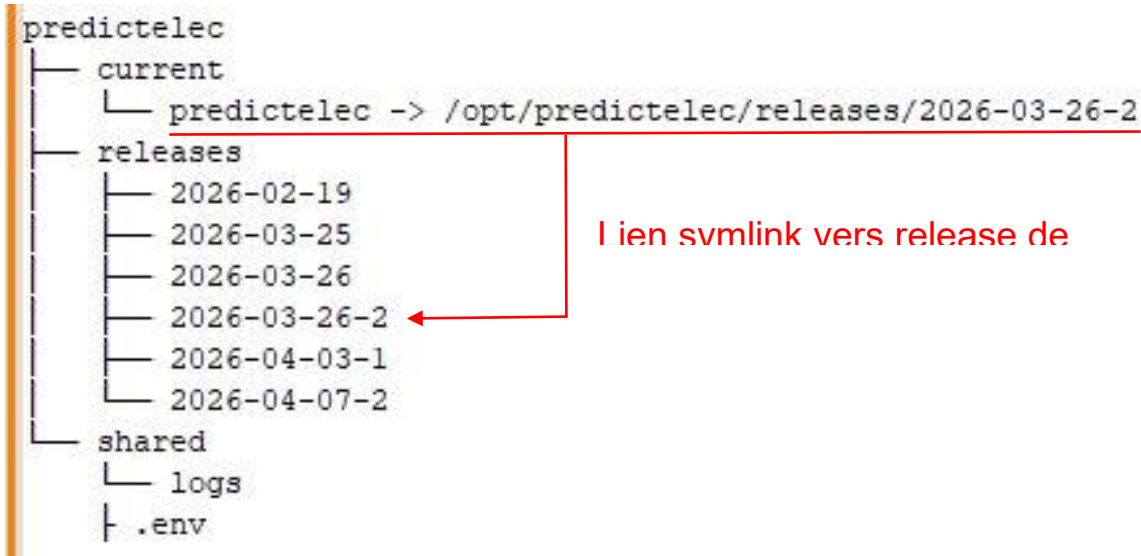
- des courbes temporelles ;
- des graphiques de production cumulée permettant des comparaisons par région.

7. Déploiement

Le code est structuré en couches avec logique métier faisant appel à des services utilisant les modèles et les APIs concernés pour ingérer et sauvegarder les données après normalisation via la couche d'accès à la base PostgreSQL.



Le déploiement repose sur un système de releases versionnées avec un lien symbolique *current* permettant un rollback rapide.



8. Monitoring

Une collecte de métriques avec Prometheus est mise en place sur les services suivants :

- Prometheus lui-même
- Cadvisor pour la surveillance des ressources utilisées par les conteneurs
- Node-exporter pour la surveillance des ressources système
- Postgres-exporter pour la surveillance de la base de données du projet

Grafana est utilisé pour la génération des dashboards. 3 dashboards sont générés:

- Docker Monitoring
- System Monitoring
- Production Database Monitoring

Prometheus, l'AlertManager de Prometheus, Node Exporter, CAD Advisor, Grafana et Postgres-exporter sont démarrés dans des conteneurs dédiés.

9. Bilan et optimisations

9.1. Principales difficultés et leçons

- La compréhension des API Météo France a nécessité un temps significatif. En effet les API proposées sont nombreuses et elles ont chacune leurs propres limitations. Il est important de ne pas sous-estimer l'effort nécessaire à l'appropriation des API pour la collecte de données.
- Les limitations propres aux API nous ont contraint à réaliser des scripts ETL relativement complexes (gestion des données consolidées versus données temps réel, parallélisme des appels à l'API en raison des quotas). L'écriture de tests unitaires et d'intégration en amont aurait permis de faciliter le

débugage et de gagner du temps.

- Avoir une vision “full-stack” de l’application dès le départ en prévoyant l’ensemble des services mis en oeuvre aurait faciliter le contrôle de la phase développement et un déploiement plus simple.

9.2. Pistes d’amélioration

- Améliorer le modèle de Machine Learning :
 - Analyser à posteriori la qualité des prédictions
 - Investiguer l’impact de nouvelles features (température ambiante, ...)
- Ré-architecturer l’application avec un modèle d’architecture en services et microservices communiquant par API permettant un déploiement distribué, une maintenance et un déploiement plus industriels de l’application.